

SHERPA.AI PRIVACY-PRESERVING MULTI-PARTY ENTITY ALIGNMENT WITHOUT INTERSECTION DISCLOSURE FOR NOISY IDENTIFIERS

Sherpa.ai

research@sherpa.ai

Abstract

Federated Learning (FL) enables collaborative model training among multiple parties without centralizing raw data. There are two main paradigms in FL: Horizontal FL (HFL), where all participants share the same feature space but hold different samples, and Vertical FL (VFL), where parties possess complementary features for the same set of samples. A prerequisite for VFL training is privacy-preserving entity alignment (PPEA), which establishes a common index of samples across parties (alignment) without revealing which samples are shared between them. Conventional private set intersection (PSI) achieves alignment but leaks intersection membership, exposing sensitive relationships between datasets. The standard private set union (PSU) mitigates this risk by aligning on the union of identifiers rather than the intersection. However, existing approaches are often limited to two parties or lack support for typo-tolerant matching.

In this paper, we introduce the Sherpa.ai multi-party PSU protocol for VFL, a PPEA method that hides intersection membership and enables both exact and noisy matching. The protocol generalizes two-party approaches to multiple parties with low communication overhead and offers two variants: an order-preserving version for exact alignment and an unordered version tolerant to typographical and formatting discrepancies. We prove correctness and privacy, analyze communication and computational (exponentiation) complexity, and formalize a universal index mapping from local records to a shared index space. This multi-party PSU offers a scalable, mathematically grounded protocol for PPEA in real-world VFL deployments, such as multi-institutional healthcare disease detection, collaborative risk modeling between banks and insurers, and cross-domain fraud detection between telecommunications and financial institutions, while preserving intersection privacy.

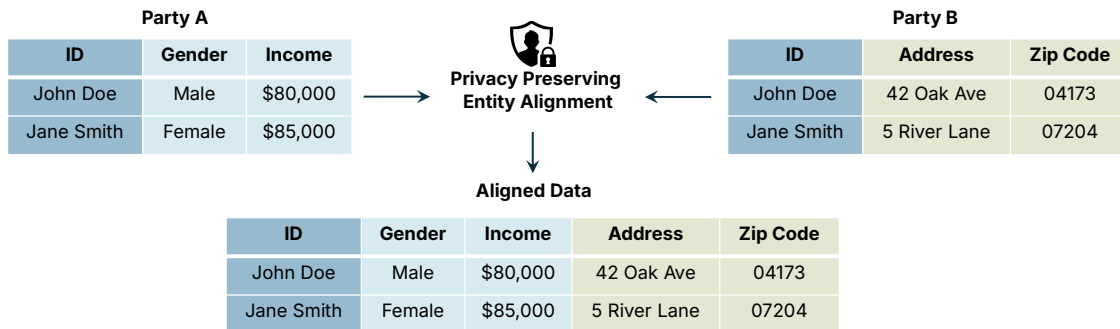


Figure 1: Illustrative example of entity alignment in VFL: based on the ID, Parties A and B perform private matching and produce Aligned Data without disclosing intersection membership or sensitive values.

1 Introduction

Federated Learning (FL) [20] enables multiple parties (nodes, clients, or devices) to jointly train a model without sharing their raw data; instead, they exchange model parameters or updates, thereby avoiding the need to centralize datasets as in standard Machine Learning (ML). FL is commonly divided into *Horizontal FL (HFL)* [34], where participants have the same features but different records, and *Vertical FL (VFL)*, where participants own complementary feature sets for an overlapping population of samples [32].

A fundamental prerequisite in VFL is that datasets be aligned *row-wise* [37]: (i) each record across parties refers to the same real-world entity, and (ii) records appear in the same order for all parties. In practice, this means establishing a common index of entities while avoiding disclosure of which entities are shared across parties (i.e., intersection membership) [30]. For example, consider a collaboration between a bank and an insurance company: the bank holds financial transaction data, while the insurer maintains policy and claim records. To jointly train a predictive model, such as estimating default risk or detecting fraud, their datasets must be aligned so that each row corresponds to the same customer across both organizations. Achieving this alignment without revealing which customers are shared between them is the challenge that privacy-preserving entity alignment (PPEA) aims to address (see Figure 1).

PPEA [9] addresses this need by aligning datasets held by different parties while preserving privacy. Two main cryptographic approaches are commonly used: private set intersection (PSI) and private set union (PSU), as illustrated in Figures 2 and 3.



Figure 2: Illustration of the PSI protocol. Only the common identifiers (IDs) between the two parties are used to form the shared training dataset.

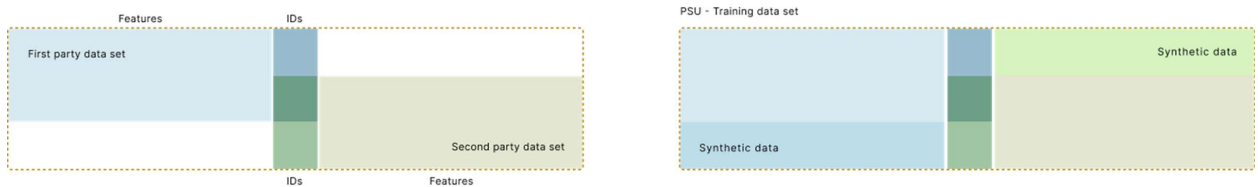


Figure 3: Illustration of the PSU protocol. All unique IDs across parties form the union dataset used for training; missing features for non-overlapping records are completed with synthetic data.

In this paper, we introduce [Sherpa.ai](#) PSU for VFL, a multi-party, union-based entity-alignment method that employs a commutative-encryption PSU protocol to conceal intersection membership and construct a shared universal index across parties. Unlike prior PSU approaches that are primarily limited to the two-party setting and exact identifier matching, our method generalizes naturally to multiple parties and supports both exact and noisy matching regimes. After alignment on the union, missing attributes can be completed using synthetic data as a common practice used in PSU for VFL training.

In VFL, we distinguish two conceptually distinct stages. First, at the alignment stage, PSU conceals intersection membership while constructing a private universal index across parties. Second, at the training stage, any downstream VFL method can operate on the aligned data produced by PSU. For example, paradigms such as [Sherpa.ai](#) Blind Vertical Federated Learning (SBVFL) [1], which replace true labels with server-generated synthetic labels and reduce party-server exchanges, may be employed to mitigate label and gradient leakage as well as communication overhead. In this sense, PSU is independent of the subsequent training method and serves as a privacy-preserving preprocessing step for VFL.

1.1 Motivation and Challenges

PSI privately computes the intersection of the parties’ identifier sets (see Figure 2) enabling them to identify which identifiers they share. However, PSI outputs the intersection of identifiers as well, which can pose a privacy risk. For example, consider a federation of a bank and a cancer clinic: learning that a particular customer of the bank is in the intersection, it implies that this individual is in the clinic’s records, potentially leaking sensitive health information (i.e., the bank could infer that the customer suffers from cancer, and hence, denying them a loan). Revealing intersection membership can therefore violate privacy [26]; this motivates union-based alignment that keeps membership hidden.

PSU computes the union of identifier sets (see Figure 3) without revealing intersection information. In PSU, the parties compute the set of all identifiers that appear in at least one dataset. By merging datasets on the union rather than the intersection, the parties avoid disclosing which identifiers they have in common. When using the union, some feature values will be missing for entities present only in one party. These gaps can be filled with synthetic data generated locally by each party, which prior work has shown can preserve model utility while improving privacy by hiding membership [26].

While several PSU protocols have been proposed in recent years, they have key limitations. Two-party designs, such as Sun et al. [26] and Tu et al. [27], target balanced or unbalanced two-party settings and, thus, do not directly scale to multi-party VFL. Multi-party or high-throughput variants (e.g., Gao et al. [7]) prioritize efficiency with heavier primitives but do not support typo-tolerant matching. Finally, differentially private unions (e.g., Gopi et al. [10]) produce approximate, rather than exact, unions. Motivated by these gaps, we develop a multi-party PSU for VFL that hides intersection membership and supports two alignment regimes: (i) an exact-hash, order-preserving regime, which is theoretically optimal when identifiers are clean and consistently formatted, and (ii) a fuzzy matching, unordered regime, which is substantially robust in real-world scenarios with noisy, heterogeneous identifier fields, but might give less accurate results in ideal conditions. In addition, some privacy-preserving entity-alignment approaches rely on a trusted coordinator or stronger trust assumptions, which may be undesirable in cross-organizational deployments; our protocol avoids this requirement. Further details and guarantees are provided in Sections 2 and 4.

1.2 Contributions

The main contributions of this work are summarized as follows:

- We propose the multi-party [Sherpa.ai](#) PSU protocol for PPEA in VFL. Unlike traditional PSI methods, our hashing-based PSU construction enables secure alignment across multiple parties without requiring a trusted third party and prevents the disclosure of intersection membership.
- We introduce an n -gram tokenization preprocessing step that enhances robustness to formatting inconsistencies and typographical errors, supporting both order-preserving (exact) and unordered (noisy) matching depending on data quality.
- We formalize the commutative encryption process based on the Diffie–Hellman key exchange principle, generalizing existing two-party PSU methods [26] to a multi-party setting with provable privacy guarantees under the semi-honest model.
- We define the procedure for computing universal indices that map each party’s local records into a shared index space, enabling subsequent data integration and joint model training.

In summary, our approach establishes a mathematically grounded and robust to noisy identifiers framework for PPEA in VFL, maintaining confidentiality while remaining practical for deployment in real-world, multi-institutional collaborations.

The remainder of this paper is organized as follows. Section 2 reviews related work. In Section 3, we formalize the problem setting for PSU in VFL. Section 4 details the proposed solution, including identifier preprocessing, hashing, PSU protocols (with or without order preservation), and synthetic data generation. Finally, Section 5 concludes the paper.

2 Related Work

Several PSU protocols have been proposed in the literature, leveraging diverse cryptographic techniques. For example, Sun et al. [26] proposed one of the first PSU solutions specifically for VFL without revealing

intersection membership. Their protocol (which we build upon) is limited to two parties; in contrast, our approach generalizes to P parties and introduces n -gram tokenization for improved matching. Gao et al. [6] developed PULSE, a parallel multi-party PSU protocol that leverages fast cryptographic operations (e.g., symmetric-key primitives and oblivious transfers) to efficiently handle large-scale datasets. Our approach shares a similar multi-party setting but prioritizes minimal communication rounds over parallel throughput. Tu et al. [27] present an enhanced two-party PSU protocol that supports both balanced and unbalanced set sizes, achieving better computational and communication performance than earlier methods. In contrast, our protocol minimizes communication overhead by utilizing commutative encryption and naturally accommodates multiple parties without requiring additional assumptions.

Beyond purely cryptographic methods, some works rely on differential privacy (DP). For instance, Gopi et al. [10] design algorithms that produce the union with rigorous privacy guarantees by injecting noise. Such DP-based approaches do not reveal exact intersection membership, but they trade off some accuracy (and typically do not yield *exact* unions) in exchange for strong privacy. Other approaches employ *homomorphic encryption* (HE): for example, Tu et al. [28] use fully HE (FHE) to compute an unbalanced PSU, achieving strong security with relatively heavy computation. Compared to these, our protocol avoids expensive public-key operations on large data, instead using hashing and modular exponentiations that are efficient, and requires only a few rounds of communication. Our primary design goal is to minimize both the number of communication rounds and the amount of exchanged data. The trade-off is that the “PSU without order” variant of our method involves a potentially expensive comparison step for handling typos, which can be mitigated through low-level implementations or optimized data structures such as Bloom filters. For additional references on the topic, we refer the reader to [17, 5, 25, 18, 14, 36, 8, 7, 15, 35].

Our approach builds upon and extends prior PSU research. In particular, our multi-party commutative encryption scheme generalizes the two-party PSU method of Sun et al. [26] by supporting any number of parties and introducing n -gram-based noisy matching. Unlike many PSI/PSU protocols that rely on heavier cryptographic tools (e.g., oblivious transfer, garbled circuits, or FHE) [16], our method keeps the computations relatively lightweight (modular exponentiations and hashing) and aims to reduce communication to two main rounds. Recent works, such as Gao et al. [6] and Tu et al. [27], focus on optimizing PSU for performance, achieving notable speed-ups through parallel operations and specialized data structures, albeit at the cost of increased protocol complexity. In contrast, our protocol emphasizes ease of integration within an FL system and simplicity of implementation, assuming semi-honest parties.

Beyond PSU-specific research, complementary lines of work are directly relevant to PPEA in VFL. Private Sample Alignment (PSA) protocols have been explored to achieve reliable multi-client VFL deployments and scalable two-party settings [33, 31], offering alternative building blocks to PSU. For asymmetric federations, differential PSI (DPSI) protects membership by adding calibrated noise to the revealed results [11]. Noisy or approximate matching has been studied in PSI for biometric search [29], while classic privacy-preserving record linkage (PPRL) methods based on n -grams and Bloom filters [24, 4] are adapted to our unordered (noisy) matching design.

A concise comparison of privacy properties across standard entity alignment (PSI), standard PSU, and our proposed [Sherpa.ai](#) PSU is presented in Table 1. Standard entity-alignment (PSI) protocols typically reveal the intersection between parties and are limited to exact, pairwise alignment, whereas standard PSU constructions hide the intersection but do not support multi-party execution and assume perfectly matching identifiers. In contrast, our proposed [Sherpa.ai](#) PSU simultaneously (i) computes only the union without revealing the intersection, (ii) naturally extends to more than two parties via a commutative-encryption design, and (iii) supports privacy-preserving noisy alignment to cope with typos and non-canonical identifiers. This combination of properties, which is crucial in realistic VFL deployments, is not provided by existing PSI/PSU-based entity-alignment schemes.

	Standard Entity Alignment (PSI)	Standard PSU	Sherpa.ai PSU
Raw identifiers exchanged	✓ No	✓ No	✓ No
Intersection revealed	✗ Yes	✓ No	✓ No
Multi-party	✓ Yes	✗ No	✓ Yes
Noisy alignment	✗ No	✗ No	✓ Yes

Table 1: Comparison of the privacy properties of different entity alignment protocols.

3 Problem Formulation

We consider a set of P parties wishing to perform VFL. Without loss of generality, assume there are $P - 1$ *passive parties*

$$\mathcal{F}_0, \dots, \mathcal{F}_{P-2}$$

and an *active party*

$$\mathcal{F}_{P-1}.$$

For each $k \in \{0, \dots, P-1\}$, party $\mathcal{F}_k$ owns a dataset $\mathcal{D}_k = \{\vec{x}_k^i\}_{i \in \{0, \dots, N_k-1\}}$ consisting of N_k data examples (rows), where each example (or row) $\vec{x}_k^i \in \mathbb{R}^{d_k}$, $N_k \in \mathbb{N} \setminus \{0\}$ and $d_k \in \mathbb{N} \setminus \{0\}$.

All parties agree on a common set of one or more identifying features that will be used for record matching (for example, a combination of name, phone number, and address). We denote by d_{match} the number of such identifying features (this d_{match} is the same for every party). Let

$$\pi_k : \mathbb{R}^{d_k} \longrightarrow \mathbb{R}^{d_{\text{match}}},$$

be the projection that extracts the d_{match} identifying attributes from party $\mathcal{F}_k$'s feature space. Each party can then derive from its dataset a set of **raw identifiers** (one identifier per example) as

$$I_{\text{raw},k} := \{\pi_k(\vec{x}_k^i) \mid k = 0, \dots, N_k - 1\}.$$

for $k = 0, \dots, P-1$. In other words, $I_{\text{raw},k}$ is the set of identifier tuples (such as personal data fields) for the records held by party $\mathcal{F}_k$. Our goal is to perform the following steps in a privacy-preserving manner, *without* revealing to any party which identifiers are shared or not shared with other parties.

1. **Compute the Union of Identifiers.** Determine the union of all parties' identifier sets:

$$\mathcal{U} := \bigcup_{k=0}^{P-1} I_{\text{raw},k}. \quad (3.1)$$

Let $N = |\mathcal{U}|$ be the total number of unique identifiers across all parties.

2. **Define Universal Indices.** Assign each identifier in the union $\mathcal{U}$ a unique **universal index**. Let $V := \{0, \dots, N-1\}$ be the set of universal indices. This is essentially an indexing of the union $\mathcal{U}$.
3. **Align Local to Universal Index Mappings.** For each party $\mathcal{F}_k$, determine a mapping

$$\varphi_k : \{0, \dots, N_k - 1\} \longrightarrow V, \quad (3.2)$$

which maps each **local index** i (the position of a record in party $\mathcal{F}_k$'s dataset) to the corresponding universal index in V . In other words, if $\pi_k(\vec{x}_k^i) \in \mathcal{U}$ ends up as the j -th element of the union, then $\phi_k(i) = j$. This mapping ϕ_k allows each party to translate its local records into the unified indexing of the union.

All of the above should be achieved without any party learning which of its identifiers were or were not present in other parties' datasets (thus, intersection membership remains private). Next, we describe our solution to this problem.

4 Our Proposed PSU Solution

Our pipeline proceeds in four stages (see Figure 4), to enable VFL PPEA without revealing intersection membership:

1. **n -gram Tokenization of Identifiers:** We first split each identifier in $I_{\text{raw},k}$ (which may be a composite of multiple fields) into a set of overlapping substrings of length n , known as n -grams. This helps standardize the format and allows for partial matching to accommodate error tolerance.
2. **Hashing of Identifiers:** The n -grams are hashed to fixed-length values. Hashing ensures a uniform representation (e.g., fixed-size bit strings or integers) for tokens, and provides a layer of one-way protection so that plaintext identifier values are not directly used in the cryptographic protocol.
3. **PSU Protocol:** Next, we perform a multi-party PSU protocol on the sets of hashed n -grams to obtain the union and the index mappings. Our PSU protocol uses commutative encryption in the cyclic group of quadratic residues to hide identifier values and achieve the required privacy properties.

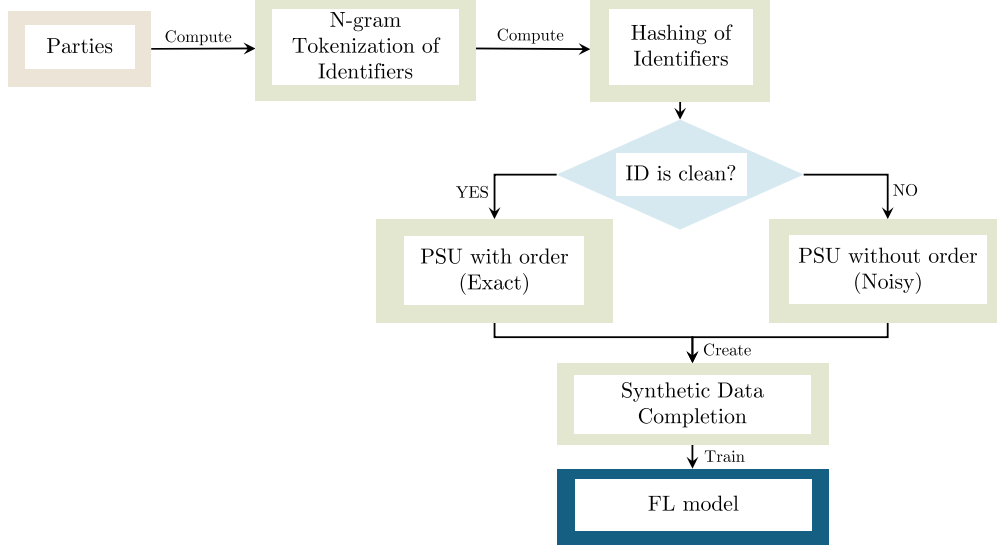


Figure 4: Pipeline of the proposed PSU protocol for multi-party VFL.

4. **Synthetic Data Completion:** After aligning on the union and obtaining universal indices, each party fills missing feature values for non-overlapping entities using privacy-preserving synthetic data generators, enabling downstream FL training without sharing raw data.

The algorithms we propose are a generalization of [26, Algorithm 1, page 5], with the following additional features.

- They work for P parties, where $P \geq 2$.
- Different identifier features are treated separately.
- Each identifier’s feature is split in n -grams of assigned length.

Conceptually, we distinguish two cases for the alignment problem. First, we consider alignment with exact hashes, where each identifier is represented as an ordered sequence of n -grams and two records are deemed equal only if all tokens match in the same order (Section 4.3). This regime is theoretically optimal when all parties store clean, consistently formatted identifiers, but it is less realistic in practice, where fields may be split, reordered, or contain typos. Second, we introduce a noisy matching regime that operates on unordered multisets of n -grams (Section 4.4). This “without order” variant is not exact even under ideal conditions, but in practice gives a very good approximation while being substantially more robust to noisy and heterogeneous identifier fields. Both cases share the same cryptographic PSU core; they differ only in how identifiers are tokenized and compared.

In general, the PSU protocol is designed and analyzed in the group of quadratic residues modulo a safe prime. Specifically, let p be a **safe prime**, i.e., p is prime and $(p-1)/2$ is also prime. Denote by $\mathbb{Z}_p = \{[0]_p, [1]_p, \dots, [p-1]_p\}$ the ring of integers mod p , and by $\mathbb{Z}_p^* = \{[1]_p, [2]_p, \dots, [p-1]_p\}$ the multiplicative group of integers mod p . We define the group of quadratic residues modulo p as

$$QR(\mathbb{Z}_p^*) := \{[x]_p \mid \exists y \in \mathbb{Z}, y^2 \equiv x \not\equiv 0 \pmod{p}\}. \quad (4.1)$$

In other words, $QR(\mathbb{Z}_p^*)$ is the subgroup of $\mathbb{Z}_p^*$ consisting of all non-zero squares mod p . By using the properties of the additive group $(\mathbb{Z}_p, +)$ and the cyclic group $(\mathbb{Z}_p^*, *)$, it is possible to prove that $(QR(\mathbb{Z}_p^*), *)$ is a group of order $\frac{p-1}{2}$. The security of the algorithms is based on the decisional Diffie-Hellman assumption [3], which stipulates that in the discrete group (4.1), performing the power (encryption) is easy, whereas performing the logarithm (decryption) is hard. As such, for our algorithms, all the operations will be taken modulo p or $q = \frac{p-1}{2}$, where p is a safe prime number.

4.1 n -gram Tokenization of Identifiers

For each party $\mathcal{F}_k$ and each raw identifier $\text{id} \in I_{\text{raw},k}$, the identifier is decomposed into n -grams. An identifier id may be a single string (e.g., a customer ID) or a tuple of attributes (e.g., first name, last name, address, etc.).

Formally, for every $k = 0, \dots, P-1$, party $\mathcal{F}_k$ possesses an identifiers set $I_{\text{raw},k}$. A raw identifier $\text{id} \in I_{\text{raw},k}$ is a vector

$$\text{id} = (\text{id}_1, \dots, \text{id}_{d_{\text{match}}}) \in \mathbb{R}^{d_{\text{match}}}.$$

Without loss of generality, assume each component id_r can be represented as a string (we can stringify numeric fields as well):

$$\mathcal{S} := \{s \mid s \text{ Python string}\}.$$

We can cast in-place each component of the identifier id as a Python string. Then,

$$\text{id} = (\text{id}_1, \dots, \text{id}_{d_{\text{match}}}) \in \mathcal{S}^{d_{\text{match}}}.$$

All parties $\mathcal{F}_0, \dots, \mathcal{F}_{P-1}$ agree on the length n_r of the n -grams in which component r of identifiers will be split, for each $r \in \{1, \dots, d_{\text{match}}\}$. Moreover, in order to perform a proper n -grams splitting, they need to agree on a length for the string of each component of identifiers: let L_r be the established length of the component r of identifiers; if $n_r > L_r$, we redefine

$$L_r \leftarrow n_r.$$

For every $k = 0, \dots, P-1$, for any identifier $\text{id} \in I_{\text{raw},k}$, for each $r \in \{1, \dots, d_{\text{match}}\}$,

- if the length of the string id_r is smaller than L_r , then $L_r - \text{length}(\text{id}_r)$ empty spaces are added;
- if the length of the string id_r is greater than L_r , then the last length $(\text{id}_r) - L_r$ characters are removed.

At this stage, we can separate id_r in $L_r - n_r + 1$ n_r -grams, using a sliding window approach. Namely, for $l \in \{0, \dots, L_r - n_r\}$, the l -th n_r -grams of id_r is

$$c_{r,l} := (\text{id}_{r,l}, \dots, \text{id}_{r,l+n_r-1}).$$

For $k = 0, \dots, P-1$, set

$$I_{\text{separated-raw},k} := \left\{ (c_{i,r,l})_{i,r,l} \mid i \in \{0, \dots, N_k - 1\}, r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\} \right\}.$$

The motivation for n -gram separation is twofold. First, it provides a flexible way to handle minor discrepancies in strings (e.g., typos or different formatting). Concretely, it allows our protocol to align identifiers such as “123 Main St.” stored as a single field with records where the street number and name are split across fields (“123” and “Main St.”), and to match “123 Main St.” against variants like “123 Main Street”, “123 main st”, or “123 Main Str”. Similarly, it is robust to differences in capitalization (“SMITH” vs. “Smith”), accent marks (“José” vs. “Jose”), and small typographical errors. Because such representations still share most of their n -grams, they are treated as near-matches in the unordered case. Second, it can improve matching accuracy by ensuring that tokens are compared at a granular level rather than via whole-string comparisons.

Our use of n -gram hashing is related to prior work on deep structured semantic models for web search, where n -gram vectors are used as inputs to neural networks, and 3-grams are found to offer a good compromise between robustness to small variations and the rate of hash collisions when order is not preserved [12]. In our setting, such noisy matching naturally induces two types of errors: (E1) false negatives, where records that truly exist in both parties are not linked, and (E2) false positives, where records belonging to different individuals are incorrectly linked. Since E2 is typically much more harmful in privacy-preserving entity resolution, we choose the similarity threshold in our matching step to strongly penalize E2-type errors, accepting a small number of E1 errors as the cost of avoiding incorrect links. Moreover, while additional neural layers on top of n -gram vectors can make direct inference of the original identifiers more difficult, they are generally designed to preserve enough information to approximately reconstruct the input and should not be seen as a primary privacy mechanism. In our protocol, n -gram noisy matching is used to enhance robustness in record linkage, while strong privacy guarantees are provided by the subsequent encryption layer.

4.2 Hashing of Identifiers

After tokenization, each party hashes its n -grams to obfuscate their values and to enable efficient cryptographic processing. This second part of our solution addresses the hashing of n -grams, mapping raw feature values into the group defined in Equation (4.1).

Our hashing procedure mainly consists of two steps:

1. Hashing by SHA3-256 and casting to `int`;
2. Projection onto $QR(\mathbb{Z}_p^*)$.

The composition of the above two operations defines a mapping:

$$h : \mathcal{S} \longrightarrow QR(\mathbb{Z}_p^*),$$

where

$$\mathcal{S} := \{s \mid s \text{ Python string}\}.$$

For every $k = 0, \dots, P - 1$, the hashed identifiers set will be denoted by I_k . We have

$$I_k = \left\{ h(c) \mid c \in I_{\text{separated-raw},k} \right\}.$$

Hashing provides a layer of privacy (an adversary must invert the hash to guess the original token) and also standardizes the representation length of tokens. Most importantly, all parties use the **same hash function** h , so if two parties have an identical n -gram, they will end up with the same hash value. From this point onward, our protocol operates on these hashed identifiers.

4.2.1 Commutative Encryption

Our PSU protocol utilizes the group $QR(\mathbb{Z}_p^*)$, defined earlier, as the space for commutative encryption. We choose a large safe prime p (on the order of 2048 bits or larger for security) and let $q = (p - 1)/2$ (which is prime). All operations on hashed identifiers will be performed modulo p , and some index arithmetic will be modulo P (for party indices).

Figure 5 illustrates the first phase of the commutative encryption process that underpins the PSU protocol. Each party (A and B in the example) begins by hashing its identifiers and encrypting them with its local secret exponent. The encrypted identifiers are then shuffled and exchanged between the parties. Upon receiving the counterpart's encrypted set, each party re-encrypts the values with its own secret exponent and shuffles them again before returning them. This iterative, commutative exchange ensures that identifiers are doubly encrypted under both parties' keys while preserving the property that identical identifiers yield identical ciphertexts, a prerequisite for performing the secure union.

Each party $\mathcal{P}_k$ generates a secret exponent s_k (in practice, multiple exponents per party are used for different protocol phases, denoted $s_{k_1}, s_{k_2}, s_{k_3} \in \{0, \dots, q - 1\}$). We define the encryption function e_s for exponent s applied on hashed token $x \in \mathbb{Z}_p$ as:

$$e_{s, \text{one component}}(x) = x^s \pmod{p}. \quad (4.2)$$

We extend this to an identifier's tokens (across all features) by applying e_s to each token: let an identifier be represented as a tuple $(x_{r,l})$ where r is the index of the feature (from 1 to d_{match}) and l is the index of the n -gram within that feature (from 1 to $L_r - n + 1$, assuming L_r is the length of feature r for that identifier). Next, we define the following vectorized encryption functions.

Let $s \in \{0, \dots, q - 1\}$ and

$$e_s : \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right] \longrightarrow \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right]$$

$$(x_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \longmapsto (x_{r,l}^s)_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}},$$

where $\sigma : \{0, \dots, N_k - 1\} \longrightarrow \{0, \dots, N_k - 1\}$ is a random permutation.

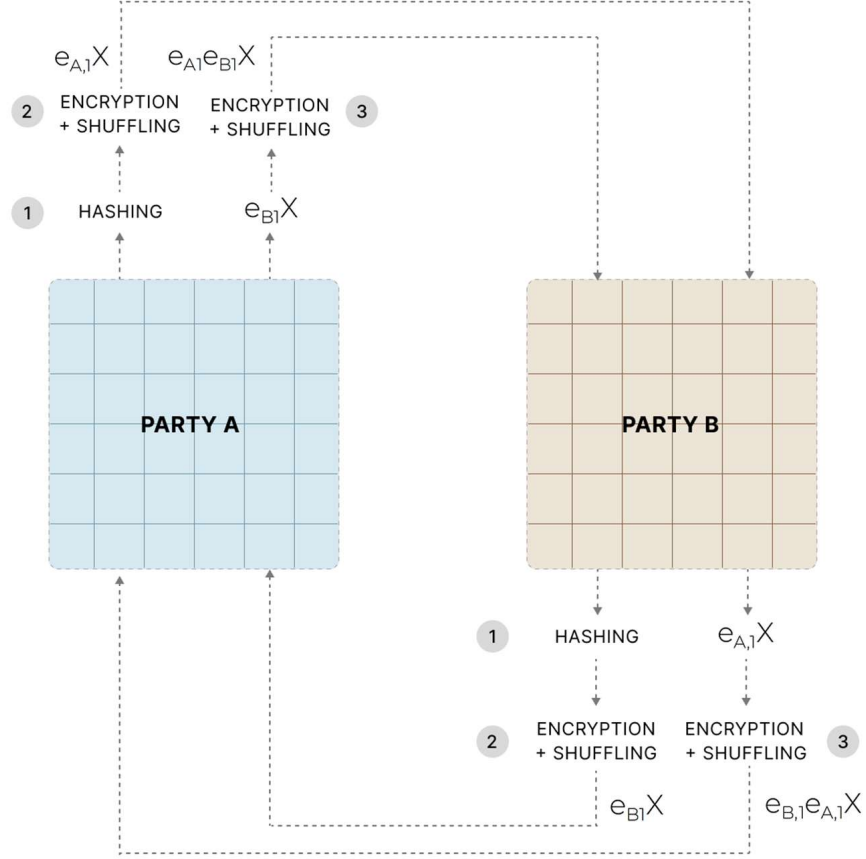


Figure 5: Scheme describing the main steps of the first part of the Diffie-Hellman protocol employed for PSU.

By the commutativity of the product in $\mathbb{Z}_p$, $e_{s_1} \circ e_{s_2} = e_{s_2} \circ e_{s_1}$, for some s_1, s_2 in $\{0, \dots, q-1\}$. Because exponentiation is applied independently to each token, two identical identifiers (same tokens in the same order) encrypted with the same exponent s yield identical outputs, while identifiers that differ in any token produce different encrypted outputs (up to hash or exponent collisions). Moreover, since $e_{s_1} \circ e_{s_2} = e_{s_2} \circ e_{s_1}$ for any $s_1, s_2 \in \{0, \dots, q-1\}$, applying exponents in any order is equivalent to a single exponentiation by $s_1 s_2 \bmod q$. This commutativity is what enables our multi-party PSU construction.

From a cryptographic standpoint, this layer is not a new primitive but the standard Diffie-Hellman exponentiation in the safe-prime subgroup $QR(\mathbb{Z}_p^*)$, i.e., the mapping $x \mapsto x^s \bmod p$ for a secret exponent $s \in \{1, \dots, q-1\}$. We deliberately use this primitive in a deterministic, commutative masking mode rather than as a randomized public-key encryption scheme. This is essential for PSU: (i) exponentiations under different parties' secret exponents must commute, so that applying all exponents in any order yields the same masked identifier, and (ii) identical identifiers must remain identical after all exponentiations, so that the server can compute the union by equality tests on the resulting masked values. Standard randomized public-key encryption schemes would typically destroy these properties, since encrypting the same value twice yields unrelated ciphertexts and encryptions under different keys do not commute. Our choice therefore, provides exactly the algebraic structure needed for multi-party PSU without a trusted third party. The privacy analysis is carried out in the semi-honest model and relies on the standard Decisional Diffie-Hellman assumption in $QR(\mathbb{Z}_p^*)$.

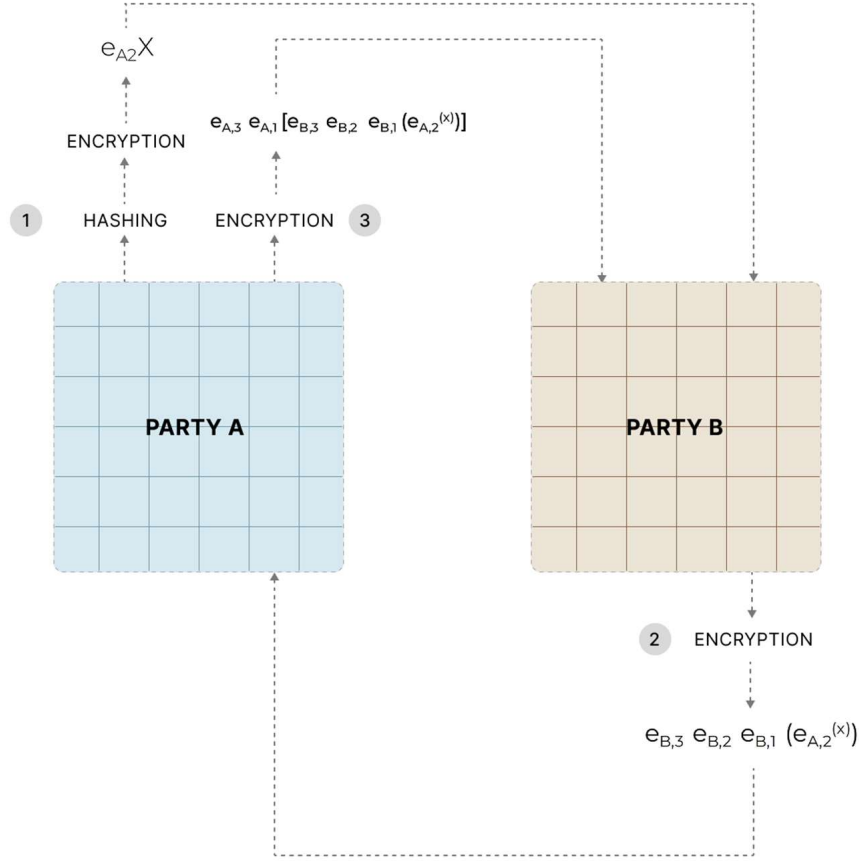


Figure 6: Scheme describing the main steps of the second part of the Diffie-Hellman protocol employed for PSU.

Figure 6 illustrates the second phase of the commutative encryption process, which determines the final mapping between local identifiers and their encrypted counterparts in the union. After the initial double-encryption round (Figure 5), each party performs additional encryption operations using its remaining secret keys and returns the results to the other party. The active party then computes the fully encrypted union and derives the mapping between each local record and the corresponding universal index. This phase finalizes the secure exchange while maintaining the commutativity property, which prevents either participant from learning the intersection membership.

Let us also define $\hat{e}_s$ as the operation of applying e_s to an entire set of identifiers (for example, an entire party's set I_k) and also randomly permuting the order of identifiers. Specifically, for every $k = 0, \dots, P - 1$,

$$\hat{e}_s : \prod_{k=0}^{N_k-1} \left\{ \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r-n_r+1} \mathbb{Z}_p \right] \right\} \rightarrow \prod_{k=0}^{N_k-1} \left\{ \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r-n_r+1} \mathbb{Z}_p \right] \right\}$$

$$(x_{i,r,l})_{i \in \{0, \dots, N_k-1\}, r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r-n_r+1\}} \mapsto (x_{\sigma(i),r,l}^s)_{i \in \{0, \dots, N_k-1\}, r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r-n_r+1\}},$$

where $\sigma : \{0, \dots, N_k - 1\} \rightarrow \{0, \dots, N_k - 1\}$ is a permutation.

We allow each party to re-index (shuffle) its set after encryption to hide any information that could be inferred from the ordering of identifiers (for example, if one party's dataset is sorted in some way, a shuffle breaks that correlation). We denote by σ a random permutation on the index set $\{0, \dots, N_k - 1\}$ used by party $\mathcal{F}_k$ during encryption. The commutativity still holds: applying $\hat{e}_{s_1}$ and then $\hat{e}_{s_2}$ (with independent random permutations) to a vector of identifiers yields the same vector as $\hat{e}_{s_2}$ followed by $\hat{e}_{s_1}$ – only the overall order might differ, but since we treat the final results as sets, order does affect correctness.

Let us now define the notion of the product of encryption functions. Let $\{s_1, \dots, s_n\} \subset \{0, \dots, q-1\}$, for some $n \in \mathbb{N} \setminus \{0\}$. Let

$$\prod_{i=1}^n e_{s_i} = e_{s_n} \circ \dots \circ e_{s_1}$$

and

$$\prod_{i=1}^n \hat{e}_{s_i} = \hat{e}_{s_n} \circ \dots \circ \hat{e}_{s_1}.$$

Whenever the index range is empty, i.e., when $n_1 > n_2$ in an expression of the form $\prod_{i=n_1}^{n_2} e_{s_i}$, the corresponding composition is defined to be the identity map.

Since the composition of encryption functions is commutative, the product operator $\prod$ defined above is well-posed. Using this commutative encryption scheme, we can construct Algorithm 1, which outlines the PSU protocol with order preservation.

4.3 PSU Protocol with Order (Exact)

We now describe the order-preserving variant of our PSU protocol. This corresponds to the exact-hash alignment regime described in the previous Section. In this case, each identifier $id \in I_{\text{raw},k}$ is first tokenized into n -grams per feature and hashed into $I_k \subset \prod_{r=1}^{d_{\text{match}}} \prod_{l=1}^{L_r - n_r + 1} QR(\mathbb{Z}_p^*)$ as in Section 4.2; the relative order of n -grams within each feature is treated as semantically meaningful. The goal is to compute a *universal set of encrypted identifiers* U that represents the union of all parties' (ordered) hashed identifiers while revealing nothing about intersection membership, together with local mappings $\varphi_k : I_{\text{raw},k} \rightarrow V$ that align each party's records to the universal index set V .

Algorithm 1 depicts the pseudocode of our solution. At a high level, it proceeds in three phases:

1. **Key setup and first commutative pass.** Each party $\mathcal{F}_k$ samples exponents $s_1^k, s_2^k, s_3^k \in \{0, \dots, q-1\}$. Using the commutative mapping $e_s(\cdot) = (\cdot)^s \bmod p$ and its setwise/shuffled lift $\hat{e}_s$, the parties cyclically apply $\hat{e}_{s_1^k}$ to every I_{k_1} , yielding $(\prod_l \hat{e}_{s_1^l}) I_{k_1}$ at the initiator $\mathcal{F}_{k_1}$ without exposing which tokens are shared.
2. **Provisional union and re-randomization.** The active party $\mathcal{F}_{P-1}$ forms the duplicate-free provisional union $I_{\text{un,prov}} = \bigcup_k (\prod_l \hat{e}_{s_1^l}) I_k$ and encodes it by applying $\prod_l \hat{e}_{s_3^l s_2^l}$, producing the final *encrypted universal identifiers* $U = (\prod_l \hat{e}_{s_3^l s_2^l}) I_{\text{un,prov}}$, which are then broadcast to all parties.
3. **Private matching and index assignment.** Each $\mathcal{F}_k$ locally re-encrypts its own I_k with $e_{s_2^k}$ and relays the result through a second commutative pass of exponents $\{s_1^\ell, s_2^\ell, s_3^\ell\}$. By commutativity, every element of I_k is transformed into the same ciphertext as its counterpart in U , enabling $\mathcal{F}_k$ to determine φ_k by testing membership in U *without* revealing whether any particular identifier belongs to the intersection.

The ordered treatment of tokens ensures that two identifiers match if and only if all their feature-wise n -gram sequences coincide, yielding exact row alignment when inputs are consistently formatted. Security follows from applying only exponentiations in the subgroup $QR(\mathbb{Z}_p^*)$ under the decisional Diffie–Hellman assumption, together with full-set shuffling $\hat{e}_s$ and a final joint re-randomization, which hides intersection membership while preserving the union. The dominant cost arises from modular exponentiations during the two commutative passes; as noted below, these operations are parallel and can be executed efficiently in a multi-threaded or distributed manner.

Algorithm 1: PSU protocol, with order.

Data: For every $k = 0, \dots, P-1$, a set of hashed identifiers $I_k \subset \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} QR(\mathbb{Z}_p^*) \right]$, with p prime and $q := \frac{p-1}{2}$ prime. All the operations on identifiers will be taken modulo p . All the operation on indices $k \in \{0, \dots, P-1\}$ will be taken modulo P .

Result: Set V of universal indices (UI) and, for any $k = 0, \dots, P-1$, map

$$\varphi_k : I_{\text{raw},k} \longrightarrow V$$

associating each identifier $\text{id} \in I_{\text{raw},k}$ (of the local dataset $\mathcal{D}_k$) to the corresponding index in the set of universal indices (UI) V .

Initialization.

for $k \leftarrow 0$ **to** $P-1$ **do**

 party $\mathcal{F}_k$ randomly generates three secret exponents s_1^k, s_2^k and s_3^k in $\{0, \dots, q-1\}$. These will be used in different rounds of encryption.

First Round – Commutative Encryption of Local Sets.

for $k_1 \leftarrow 0$ **to** $P-1$ **do**

 party $\mathcal{F}_{k_1}$ computes

$$\widehat{e}_{s_1^{k_1}} I_{k_1},$$

 and sends it to the next party $\mathcal{F}_{k_1+1}$ (indices mod P so that e.g. $\mathcal{F}_P$ is $\mathcal{F}_0$).

We now start an additional encryption pipeline, where each subsequent party $\mathcal{F}_{k_2}$ (for $k_2 = k_1 + 1$ up to $P-1$ and then wrapping around to $0, \dots, k_1-1$) encrypts as follows.

for $k_2 \leftarrow 0, k_2 \neq k_1$ **to** $P-1$ **do**

 When party $\mathcal{F}_{k_2}$ receives $\left[\prod_{l=0, l \neq k_1}^{k_2-1} \widehat{e}_{s_1^l} \right] \widehat{e}_{s_1^{k_1}} I_{k_1}$, it computes

$$\left[\prod_{l=0, l \neq k_1}^{k_2} \widehat{e}_{s_1^l} \right] \widehat{e}_{s_1^{k_1}} I_{k_1},$$

 and sends it to $\mathcal{F}_{k_2+1}$.

 When $\mathcal{F}_{k_1}$ receives back $\prod_{l=0}^{P-1} \widehat{e}_{s_1^l} I_{k_1}$, it stops and sends it to $\mathcal{F}_{P-1}$.

Union Computation (Provisional).

$\mathcal{F}_{P-1}$ performs the union (repetitions must be avoided)

$$I_{\text{un,prov}} := \bigcup_{k=0}^{P-1} \prod_{l=0}^{P-1} \widehat{e}_{s_1^l} I_k.$$

Now the active party $\mathcal{F}_{P-1}$ orchestrates a second round of encryption on the provisional union to further blind the values.

for $k \leftarrow P-1$ **to** 0 **do**

 party $\mathcal{F}_k$ computes

$$\prod_{l=k}^{P-1} \widehat{e}_{s_3^l s_2^l} I_{\text{un,prov}},$$

 and sends it to $\mathcal{F}_{k-1}$.

Once $\mathcal{F}_{P-1}$ receives

$$\prod_{l=0}^{P-1} \widehat{e}_{s_3^l s_2^l} I_{\text{un,prov}},$$

party $\mathcal{F}_{P-1}$ determine U , the set of encrypted universal identifiers (UID)

$$U := \prod_{l=0}^{P-1} \widehat{e}_{s_3^l s_2^l} I_{\text{un,prov}}.$$

Algorithm 1: PSU protocol, with order (continued).

for $k \leftarrow 0$ **to** $P - 1$ **do**

└ Party $\mathcal{F}_{P-1}$ sends the set of encrypted universal identifiers (UID) to party $\mathcal{F}_k$.

Private Matching of Identifiers.

Finally, each party determines the mapping ϕ_k between its local identifiers and the universal set. This is done by each party independently (in parallel) using the secret exponents and the encrypted data.

for $k_1 \leftarrow 0$ **to** $P - 1$ **do**

└ for any $x \in I_{k_1}$, party $\mathcal{F}_{k_1}$ computes $e_{s_2^{k_1}} x$ and sends it to $\mathcal{F}_{k_1+1}$.

└ **for** $k \leftarrow 1$ **to** $P - 1$ **do**

└└ When $\mathcal{F}_{k_1+k}$ receives

$$e_{s_2^{k_1}} \prod_{l=1}^{k-1} \left(e_{s_3^{k_1+l} s_2^{k_1+l} s_1^{k_1+l}} \right) x,$$

└└ it computes

$$e_{s_2^{k_1}} \prod_{l=1}^k \left(e_{s_3^{k_1+l} s_2^{k_1+l} s_1^{k_1+l}} \right) x,$$

└└ and sends it to $\mathcal{F}_{k_1+k}$. When $\mathcal{F}_{k_1}$ receives back

$$e_{s_2^{k_1}} \prod_{k_2=0, k_2 \neq k_1}^{P-1} \left(e_{s_3^{k_2} s_2^{k_2} s_1^{k_2}} \right) x,$$

└└ party $\mathcal{F}_{k_1}$ computes

$$\prod_{k_2=0}^{P-1} \left(e_{s_3^{k_2} s_2^{k_2} s_1^{k_2}} \right) x$$

└└ and party $\mathcal{F}_{k_1}$ stores

$$\begin{aligned} \tilde{\varphi}_{k_1} : I_{k_1} &\longrightarrow U \\ x &\longmapsto \prod_{k_2=0}^{P-1} \left(e_{s_3^{k_2} s_2^{k_2} s_1^{k_2}} \right) x \end{aligned}$$

└└ and determines

$$\varphi_{k_1} : I_{k_1} \longrightarrow V.$$

At the end of Algorithm 1, all parties share a common indexing of the union of identifiers, and none of them has learned which identifiers are exclusively held by which party. The intersection information is protected because any identifier in the intersection appears in U just like any other, with no party knowing if it came from one or multiple datasets.

Complexity. The first round of the algorithm involves each party’s dataset traversing the network of P parties, resulting in P transmissions per dataset (total transmissions of P^2 in the worst case). The second round similarly involves P transmissions of the union (which size N could be larger than individual set sizes). The final matching step involves each identifier being sent through P parties (so $N_k \times P$ operations for party k). The computational cost is dominated by the modular exponentiations on possibly large sets; however, these are commutative (no interactive OT or public-key operations per item beyond exponentiation). Since the modular exponentiations on each token are independent, the protocol can be efficiently parallelized across threads or distributed parties.

In large-scale deployments, a lightweight pre-alignment or “blocking” stage could be introduced before the cryptographic protocol to reduce the number of candidate comparisons. For example, each party could exchange salted hash summaries (e.g., SHA-256) of identifiers to exclude obviously non-matching entries, thereby decreasing computational load while preserving privacy.

4.4 PSU Protocol without Order (Noisy)

Next, we describe the variant of the protocol that does not preserve the order of n -grams within each identifier’s features. This corresponds to the more practical “noisy matching” alignment regime, designed to handle noisy and heterogeneous identifier fields. This “without order” PSU variant is designed to tolerate

typographical variations or inconsistencies in identifiers by treating each as an unordered multiset of tokens. In this case, two identifiers can be considered a match (representing the same entity) even if their tokens are in a different order or one identifier has an extra token that the other lacks, as long as a majority of tokens overlap. This is essentially a private noisy matching of identifiers.

The overall structure of the protocol remains similar to the with-order case, but there are two key differences.

- In the encryption steps, we introduce an additional random permutation of token positions within each identifier's feature. Previously, e_s mapped each token but kept its position fixed in its feature. Now, we modify e_s to also randomly permute the positions of the n -grams in each feature (or use a fixed permutation ν_r per exponent) so that the token order information is eliminated (see Definition (4.3)).
- In the union matching step, because tokens are now unordered, we cannot simply take identical encrypted identifiers as an one-to-one match. Two identifiers that represent the same entity might not encrypt to an identical tuple if one had an extra token or tokens were in a different order originally. To address this, we implement a special comparison sub-protocol (Algorithm 2) which privately tests if two encrypted identifiers approximately match, given a tolerance threshold λ . This comparison algorithm essentially counts the number of encrypted tokens two identifiers have in common and determines that they are the same if a sufficiently large fraction of tokens match.

We introduce a threshold parameter $0 < \lambda \leq 1$ which governs the matching criterion. For each feature r , let $L_r - n + 1$ be the total number of n -grams for that feature in a fully formatted identifier (assuming no missing tokens). We assume that each identifier has at least $\lceil \lambda(L_r - n + 1) \rceil$ tokens for feature r (this is reasonable if λ is, say, 0.8, ensuring we only consider matches if both have a significant portion of the full token set). Algorithm 2 takes two encrypted identifiers (each a set of tokens per feature) and returns 1 if they are deemed a match (same entity) or 0 otherwise, without revealing any additional information.

Let $s \in \{0, \dots, q-1\}$ and

$$e_s : \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right] \rightarrow \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right] \quad (4.3)$$

$$(x_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \mapsto (x_{r, \nu_r(l)}^s)_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}}$$

and, for every $k = 0, \dots, P-1$,

$$\hat{e}_s : \prod_{j=0}^{N_k-1} \left\{ \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right] \right\} \rightarrow \prod_{j=0}^{N_k-1} \left\{ \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right] \right\}$$

$$(x_{j,r,l})_{j \in \{0, \dots, N_k-1\}, r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \mapsto (x_{\sigma(j), r, \nu_r(l)}^s)_{j \in \{0, \dots, N_k-1\}, r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}}$$

with $\nu_r : \{1, \dots, L_r - n_r + 1\} \rightarrow \{1, \dots, L_r - n_r + 1\}$ and $\sigma : \{0, \dots, N_k - 1\} \rightarrow \{0, \dots, N_k - 1\}$ random permutations (the index $r \in \{1, \dots, d_{\text{match}}\}$).

Here ν_r is a permutation of the positions for feature r . This essentially jumbles the token order for each feature before applying the exponent. Each party can choose a random ν_r when applying its exponent, or a deterministic one, such as sorting by token value; the important part is that the order is not preserved through encryption.

By the commutativity of the product in $\mathbb{Z}_p$, $e_{s_1} \circ e_{s_2} \simeq e_{s_2} \circ e_{s_1}$, for some s_1, s_2 in $\{0, \dots, q-1\}$, the symbol $\simeq$ meaning equality up to a permutation of the n -grams. Namely, for $r \in \{1, \dots, d_{\text{match}}\}$, there exist a permutation $\omega_r : \{1, \dots, L_r - n_r + 1\} \rightarrow \{1, \dots, L_r - n_r + 1\}$, such that, for any

$$(x_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \in \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right],$$

we have

$$e_{s_1} \circ e_{s_2} \left((x_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \right) = e_{s_2} \circ e_{s_1} \left((x_{r, \omega_r(l)})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \right) \quad (4.4)$$

$$\left((x_{i,j,r,l})_{i \in \{0, \dots, n\}, j \in \{0, \dots, N_k - 1\}, r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \right) \mapsto (\tilde{x}_{j,r,l})_{j \in \{0, \dots, \hat{N} - 1\}, r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} ,$$

Let $\{s_1, \dots, s_n\} \subset \{0, \dots, q - 1\}$, for some $n \in \mathbb{N} \setminus \{0\}$. Define

$$\prod_{i=1}^n e_{s_i} = e_{s_n} \circ \dots \circ e_{s_1}$$

and

$$\prod_{i=1}^n \hat{e}_{s_i} = \hat{e}_{s_n} \circ \dots \circ \hat{e}_{s_1}.$$

In products $\prod_{i=n_1}^{n_2} \dots$, if $n_1 > n_2$, the result is defined as the identity.

Let $\hat{N} := \sum_{k=0}^{n+1} N_k$. We define the concatenation operator as

$$\Lambda : \prod_{k=0}^{P-1} \left\{ \prod_{j=0}^{N_k-1} \left[\prod_{r=1}^{d_{\text{match}}} \left(\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right) \right] \right\} \longrightarrow \prod_{j=0}^{\hat{N}-1} \left[\prod_{r=1}^{d_{\text{match}}} \left(\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right) \right]$$

and $\tilde{x}_{j,r,l} = x_{i,j,r,l}$, by setting $\hat{j} := j - \sum_{k=0}^{j-1} N_k$ and for

$$\begin{aligned} k &\in \{0, \dots, P - 1\}, \\ j &\in \left\{ \sum_{k=0}^{j-1} N_k, \dots, \sum_{k=0}^j N_k - 1 \right\}, \\ r &\in \{1, \dots, d_{\text{match}}\}, \\ l &\in \{1, \dots, L_r - n_r + 1\}. \end{aligned}$$

Before defining the main Algorithm 3, we need to define an algorithm for comparing two encrypted identifiers as shown below.

Algorithm 2: Encrypted Identifier Comparison.**Data:** Encrypted identifiers

$$(x_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, p_{1,r}\}} \in \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{p_{1,r}} \mathbb{Z}_p \right]$$

and

$$(y_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, p_{2,r}\}} \in \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{p_{2,r}} \mathbb{Z}_p \right].$$

Threshold $\lambda \in (0, 1]$. Assume $p_{i,r} \in [\lceil \lambda L_r - n_r + 1 \rceil, L_r - n_r + 1]$, for $r \in \{1, \dots, d_{\text{match}}\}$.**Result:** 1 if the encrypted identifiers match. 0 otherwise. If 1, return $\text{list}_{\text{match}}$.*Initialization.*

Set

$$\text{list}_{\text{match}} \leftarrow [].$$

*Comparison.***for** $r \leftarrow 1$ **to** d_{match} **do**

Set

$$\text{list}_{\text{match},r} \leftarrow [].$$

for $l_1 \leftarrow 0$ **to** $p_{1,r}$ **do****for** $l_2 \leftarrow 0$ **to** $p_{2,r}$ **do**

If

$$x_{r,l_1} = y_{r,l_2}$$

and l_1 does not belong to $\text{list}_{\text{match},r}$, append l_1 to $\text{list}_{\text{match},r}$.If the cardinality of $\text{list}_{\text{match},r}$ is greater than

$$\lceil \lambda L_r - n_r + 1 \rceil,$$

return 0. Else, append $\text{list}_{\text{match},r}$ to $\text{list}_{\text{match}}$.Return 1 and $\text{list}_{\text{match}}$.**Remark 4.1** (Equivalence). *Observe that algorithm (2) defines a relation between*

$$\prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{p_{1,r}} \mathbb{Z}_p \right]$$

and

$$\prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{p_{2,r}} \mathbb{Z}_p \right],$$

where two respective elements

$$x := (x_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, p_{1,r}\}}$$

and

$$y := (y_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, p_{2,r}\}}$$

are related if algorithm (2) applied to (x, y) returns 1. However, this relation may not be an equivalence relation, since it may not be transitive.

An adaptive selection of the matching threshold λ could further improve PPEA accuracy by adjusting it according to data quality or field variability. For instance, domain-specific calibration (e.g., different λ values for names and addresses) or automatic tuning using validation data could balance recall and precision in noisy matching without altering the cryptographic design. Further extensions could integrate alternative similarity metrics, such as edit distance or locality-sensitive hashing (LSH) [13], enabling matching beyond token overlap while maintaining privacy-preserving properties.

Next, we present the main algorithm for our PSU without order.

Algorithm 3: PSU protocol, without order.

Data: For every $k = 0, \dots, P-1$, a set of hashed identifiers $I_k \subset \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} QR(\mathbb{Z}_p^*) \right]$, with p prime and $q := \frac{p-1}{2}$ prime. All the operations on identifiers will be taken modulo p . All the operation on indices $k \in \{0, \dots, P-1\}$ will be taken modulo P . Threshold $\lambda \in (0, 1]$.

Result: Set V of universal indices (UI) and, for any $k = 0, \dots, P-1$, map

$$\varphi_k : \{0, \dots, N_k - 1\} \longrightarrow V$$

associating each index i of the local dataset $\mathcal{D}_k$ to the corresponding index in the set of universal indices (UI) V .

Initialization.

for $k \leftarrow 0$ **to** $P-1$ **do**

└ party $\mathcal{F}_k$ randomly generates s_1^k, s_2^k and s_3^k in $\{0, \dots, q-1\}$.

First Round – Commutative Encryption of Local Sets.

for $k_1 \leftarrow 0$ **to** $P-1$ **do**

└ party $\mathcal{F}_{k_1}$ computes

$$\widehat{e}_{s_1^{k_1}} I_{k_1},$$

and sends it to $\mathcal{F}_{k_1+1}$.

for $k_2 \leftarrow 0, k_2 \neq k_1$ **to** $P-1$ **do**

└ When $\mathcal{F}_{k_2}$ receives $\left[\prod_{l=0, l \neq k_1}^{k_2-1} \widehat{e}_{s_1^l} \right] \widehat{e}_{s_1^{k_1}} I_{k_1}$, it computes

$$\left[\prod_{l=0, l \neq k_1}^{k_2} \widehat{e}_{s_1^l} \right] \widehat{e}_{s_1^{k_1}} I_{k_1},$$

and sends it to $\mathcal{F}_{k_2+1}$.

└ When $\mathcal{F}_{k_1}$ receives back $\prod_{l=0}^{P-1} \widehat{e}_{s_1^l} I_{k_1}$, it stops and sends it to $\mathcal{F}_{P-1}$.

Union Computation (Provisional).

$\mathcal{F}_{P-1}$ define a vector

$$V_{\text{un,prov}} := \Lambda \left(\prod_{l=0}^{P-1} \widehat{e}_{s_1^l} I_0, \dots, \prod_{l=0}^{P-1} \widehat{e}_{s_1^l} I_k, \dots, \prod_{l=0}^{P-1} \widehat{e}_{s_1^l} I_n \right).$$

We have

$$\widehat{N} = \text{length}_{\text{first axis}}(V_{\text{un,prov}}). \quad (4.5)$$

for $k_1 \leftarrow 0$ **to** $\widehat{N}-1$ **do**

└ **for** $k_2 \leftarrow k_1$ **to** $\widehat{N}-1$ **do**

└└ $\mathcal{F}_{P-1}$ applies algorithm 2 to $V_{\text{un,prov},k_1}$, with $V_{\text{un,prov},k_2}$. If the output is 1, remove $V_{\text{un,prov},k_2}$ and remove from $V_{\text{un,prov},k_1}$ n -grams indicated in $\text{list}_{\text{match}}$.

From elements of $V_{\text{un,prov},k_1}$, party $\mathcal{F}_{P-1}$ removes randomly components to reduce length of each element to $\lceil \lambda L_r - n_r + 1 \rceil$. $\mathcal{F}_{P-1}$ defines

$$I_{\text{un,prov}} := \left\{ V_{\text{un,prov},l} \mid l \in \{1, \dots, \text{length}_{\text{first axis}}(V_{\text{un,prov}})\} \right\},$$

where $\text{length}_{\text{first axis}}(V_{\text{un,prov}})$ may have changed, with respect to (4.5), because of the removals.

for $k \leftarrow P-1$ **to** 0 **do**

└ party $\mathcal{F}_k$ computes

$$\prod_{l=k}^{P-1} \widehat{e}_{s_3^l s_2^l} I_{\text{un,prov}},$$

and sends it to $\mathcal{F}_{k-1}$.

Once $\mathcal{F}_{P-1}$ receives

$$\prod_{l=0}^{P-1} \widehat{e}_{s_3^l s_2^l} I_{\text{un,prov}},$$

Algorithm 3: PSU protocol, without order (continued).

for $i \leftarrow 1$ **to** $P - 1$ **do**

 party $\mathcal{F}_{P-1}$ sends

$$\prod_{l=0}^{P-1} \widehat{e}_{s_3^l s_2^l} I_{\text{un,prov}},$$

 to $\mathcal{F}_k$.

 Party $\mathcal{F}_{P-1}$ determines U the set of encrypted universal identifiers (UID)

$$U := \prod_{l=0}^{P-1} \widehat{e}_{s_3^l s_2^l} I_{\text{un,prov}}.$$

for $k \leftarrow 0$ **to** $P - 1$ **do**

 Party $\mathcal{F}_{P-1}$ sends the set of encrypted universal identifiers (UID) to party $\mathcal{F}_k$.

Private Matching of Identifiers.
for $k_1 \leftarrow 0$ **to** $P - 1$ **do**

 for any $x \in I_{k_1}$, party $\mathcal{F}_{k_1}$ computes $e_{s_2^{k_1}} x$ and sends it to $\mathcal{F}_{k_1+1}$.

for $k \leftarrow 1$ **to** $P - 1$ **do**

 When $\mathcal{F}_{k_1+k}$ receives

$$e_{s_2^{k_1}} \prod_{l=1}^{k-1} \left(e_{s_3^{k_1+l} s_2^{k_1+l} s_1^{k_1+l}} \right) x,$$

it computes

$$e_{s_2^{k_1}} \prod_{l=1}^k \left(e_{s_3^{k_1+l} s_2^{k_1+l} s_1^{k_1+l}} \right) x,$$

 and sends it to $\mathcal{F}_{k_1+k}$. When $\mathcal{F}_{k_1}$ receives back

$$e_{s_2^{k_1}} \prod_{k_2=0, k_2 \neq k_1}^{P-1} \left(e_{s_3^{k_2} s_2^{k_2} s_1^{k_2}} \right) x,$$

 party $\mathcal{F}_{k_1}$ computes

$$\prod_{k_2=0}^{P-1} \left(e_{s_3^{k_2} s_2^{k_2} s_1^{k_2}} \right) x. \tag{4.6}$$

 Party $\mathcal{F}_{k_1}$ checks the existence of an element $\hat{x} \in U$, having $\lceil \lambda L_r - n_r + 1 \rceil$ n -grams in common, with (4.6). If this element exists, party $\mathcal{F}_{k_1}$ stores

$$\tilde{\varphi}_{k_1} : I_{k_1} \longrightarrow U$$

$$x \longmapsto \hat{x}$$

and determines

$$\varphi_{i_1} : I_{i_1} \longrightarrow V.$$

Remark 4.2. In case the order of n -grams is not preserved, to reduce the computational cost as well as save memory, Bloom filters (bit arrays) are used to store encrypted identifiers, i.e., an encrypted identifier

$$(x_{r,l})_{r \in \{1, \dots, d_{\text{match}}\}, l \in \{1, \dots, L_r - n_r + 1\}} \in \prod_{r=1}^{d_{\text{match}}} \left[\prod_{l=1}^{L_r - n_r + 1} \mathbb{Z}_p \right], \quad (4.7)$$

is represented by a Bloom filter (bitarray) $b \in \{0, 1\}^P$ by setting

$$b(x_{r,l}) = 1, \forall (r, l) \in \{1, \dots, d_{\text{match}}\} \times \{1, \dots, L_r - n_r + 1\}$$

and 0 elsewhere. We highlight two properties of this representation:

1. Two identifiers that are equivalent according to (4.4) produce the same Bloom filter, since Bloom filters defined in (4.7) are insensitive to n -gram order.
2. Two identifiers that are not equivalent according to (4.4) may still produce the same Bloom filter, because the Bloom filters in (4.7) ignore both n -gram order and feature order. This may lead to false matches. Using a separate Bloom filter per feature would reduce this effect, but here we use a single Bloom filter for all features to save memory.

In future implementations, precision could be further improved by adopting multiple Bloom filters (one per identifier feature) or by applying multiple hash functions per token, thereby reducing false positives while maintaining reasonable memory use.

In scenarios where a subset of entities already share persistent global identifiers (e.g., national or organizational IDs), these records can be excluded from the PSU protocol and directly merged, with the alignment executed only on the remaining unmatched entities. This optimization preserves security while avoiding redundant computation in practical deployments. An additional layer of formal privacy could be incorporated by introducing DP perturbations, such as dummy identifiers or randomized mappings, to protect against membership inference in extreme asymmetric cases.

Finally, Bloom filter construction can be parallelized over data blocks, offering substantial runtime reductions for large-scale datasets, as shown in other parallel PSU implementations such as PULSE [6].

4.5 Synthetic Data Completion

During model training, each party must generate a synthetic dataset to fill in the missing features in its local data, i.e., those present in the datasets of other parties but absent locally. To this end, the Synthetic Data Vault (SDV) [21] is the state-of-the-art open-source library for generating high-quality synthetic data.

SDV uses ML models to capture the statistical properties and dependencies of real datasets, enabling the creation of synthetic data that preserves both structure and utility, while protecting individual privacy. It supports tabular, time-series, and relational data and provides tools for evaluating data fidelity and privacy. By offering privacy-compliant data, SDV facilitates secure data sharing, model testing, and training in sensitive domains such as healthcare and finance, without compromising confidentiality. Two SDV backends are particularly suitable for our purpose: Gaussian Copula and Conditional Tabular Generative Adversarial Network (CTGAN).

The Gaussian Copula backend employs statistical modeling through copulas, i.e., functions that describe dependencies between random variables, to generate synthetic data. Each feature is first fitted to an appropriate marginal distribution (e.g., Gaussian, Exponential) and normalized within a uniform range $(0, 1)$ via cumulative distribution functions (CDFs). Correlations among variables are captured by a correlation matrix that models the linear dependence between transformed variables. Synthetic samples are then generated from the fitted Gaussian Copula model and inverse-transformed back to the original feature space. By operating in a transformed space and focusing on correlations rather than exact values, this method ensures that individual-level information is not reproduced while maintaining realistic relationships among variables. Gaussian Copula is particularly effective for tabular data of moderate dimensionality, capturing non-linear dependencies with low computational cost.

The CTGAN backend extends the standard Generative Adversarial Network (GAN) architecture to tabular data with mixed types, imbalanced distributions, and/or categorical features. It consists of two neural components: (i) a *generator*, which produces synthetic records, and (ii) a *discriminator*, which distinguishes real from synthetic samples. Continuous features are normalized using min-max scaling and mode-specific

normalization, which increases representation in dense data regions, while categorical variables are transformed into binary vectors. CTGAN selects a column at random during training as a conditioning variable and samples data accordingly, allowing the generator to learn relationships between that column and the rest of the dataset. After training, the generator produces realistic synthetic rows from random noise and conditional inputs, which are then inverse-transformed to match the original data types and distributions. This approach performs particularly well when the underlying data relationships are complex and traditional statistical models fail to capture them.

From the perspective of missing-data theory, our synthetic completion step can be viewed as a form of model-based imputation, where unobserved features are generated conditional on the observed ones. Classical approaches distinguish between (i) data imputation, which replaces missing values by point estimates or draws from the posterior predictive distribution (e.g., via the EM algorithm or multiple imputation), and (ii) marginalization, where learning and inference integrate over the distribution of missing values without explicitly filling them in [2, 22, 19, 23]. Our PSU-based framework is compatible with both views: the synthetic features we generate correspond to imputations of unobserved modalities, while downstream federated models could in principle be trained in a marginalization style by averaging over multiple synthetic completions, leveraging the rich toolbox developed in the missing-data literature.

5 Conclusions

In this paper, we presented the [Sherpa.ai](#) multi-party PSU protocol for PPEA in VFL. The proposed method enables multiple parties to align their datasets without revealing intersection membership, thereby strengthening privacy guarantees in collaborative ML settings. The protocol generalizes the prior two-party PSU approaches to a multi-party scenario with low communication overhead. Two complementary variants were presented: an order-preserving version for exact alignment and an unordered version that supports noisy matching, tolerant to typographical and formatting inconsistencies. Together, these algorithms offer a flexible trade-off between precision and robustness, depending on data quality and application context.

Beyond the core design, we discussed implementation aspects, including Bloom filter representations, adaptive thresholds, and parallelization strategies, to enhance scalability. Through secure and accurate entity alignment, the proposed [Sherpa.ai](#) PSU protocol empowers organizations to collaborate on VFL applications in a privacy-preserving manner, enabling practical, privacy-preserving VFL across sensitive domains, including healthcare, finance, manufacturing, aerospace, cybersecurity, and the defense industry.

In practice, both exact and noisy PSUs can be integrated with a paradigm such as SBVFL [1], which establishes a new benchmark in privacy preservation while enhancing computational efficiency. This paradigm achieves stronger confidentiality guarantees than conventional aggregation methods, alongside faster convergence and improved scalability. SBVFL thus represents a safer and more resilient framework for the next generation of secure, privacy-preserving, and collaborative intelligence.

Contributions and Acknowledgments

Daniel M. Jimenez-Gutierrez

Dario Pighin

Enrique Zuazua

Georgios Kellaris

Joaquin Del Rio

Oleksii Sliusarenko

Xabi Uribe-Etxebarria

The authors are presented in alphabetical order by first name.

References

- [1] A. Acero, D. M. Jimenez-Gutierrez, D. Pighin, E. Zuazua, J. Del Rio, and X. Uribe-Etxebarria. The sherpa. ai blind vertical federated learning paradigm to minimize the number of communications, 2025.
- [2] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the royal statistical society: series B (methodological)*, 39(1):1--22, 1977.
- [3] W. Diffie and M. Hellman. New directions in cryptography. *IEEE transactions on Information Theory*, 22(6):644--654, 1976.
- [4] E. A. Durham, M. Kantarcioglu, Y. Xue, C. Toth, M. Kuzu, and B. Malin. Composite bloom filters for secure record linkage. *IEEE transactions on knowledge and data engineering*, 26(12):2956--2968, 2013.
- [5] K. Frikken. Privacy-preserving set union. In J. Katz and M. Yung, editors, *Applied Cryptography and Network Security*, pages 237--252, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [6] J. Gao, S. Nguyen, M. Blanton, and N. Trieu. Pulse: Parallel private set union for large-scale entities, 2025.
- [7] J. Gao, S. Nguyen, and N. Trieu. Toward a practical multi-party private set union, 2023.
- [8] Y. Gao, X. Zheng, and C. Hu. A multi-party private set union protocol against malicious adversary. In Y. Pei, H. S. Ma, Y.-W. Chan, and H.-Y. Jeong, editors, *Proceedings of Innovative Computing 2024, Vol. 4*, pages 159--167, Singapore, 2024. Springer Nature Singapore.
- [9] A. Gkoulalas-Divanis, D. Vatsalan, D. Karapiperis, and M. Kantarcioglu. Modern privacy-preserving record linkage techniques: An overview. *IEEE Transactions on Information Forensics and Security*, 16:4966--4987, 2021.
- [10] S. Gopi, P. Gulhane, J. Kulkarni, J. H. Shen, M. Shokouhi, and S. Yekhanin. Differentially private set union. In H. D. III and A. Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3627--3636, PMLR, 13--18 Jul 2020. PMLR.
- [11] Y. He, X. Tan, J. Ni, L. T. Yang, and X. Deng. Differentially private set intersection for asymmetrical id alignment. *IEEE Transactions on Information Forensics and Security*, 17:3479--3494, 2022.
- [12] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, and L. Heck. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM International Conference on Information & Knowledge Management, CIKM '13*, page 2333--2338, New York, NY, USA, 2013. Association for Computing Machinery.
- [13] P. Indyk and R. Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing, STOC '98*, page 604--613, New York, NY, USA, 1998. Association for Computing Machinery.
- [14] Y. Jia, S.-F. Sun, H.-S. Zhou, J. Du, and D. Gu. Shuffle-based private set union: Faster and more secure. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2947--2964, Boston, MA, Aug. 2022. USENIX Association.
- [15] Y. Jia, S.-F. Sun, H.-S. Zhou, and D. Gu. Scalable private set union, with stronger security. Cryptology ePrint Archive, Paper 2024/922, 2024.
- [16] D. M. Jimenez-Gutierrez, Y. Falkouskaya, J. L. Hernandez-Ramos, A. Anagnostopoulos, I. Chatzi-geannakis, and A. Vitaletti. On the security and privacy of federated learning: A survey with attacks, defenses, frameworks, applications, and future directions. *Information Fusion*, 131:104155, 2026.
- [17] L. Kissner and D. Song. Privacy-preserving set operations. In V. Shoup, editor, *Advances in Cryptology -- CRYPTO 2005*, pages 241--257, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [18] V. Kolesnikov, M. Rosulek, N. Trieu, and X. Wang. Scalable private set union from symmetric-key techniques. Cryptology ePrint Archive, Paper 2019/776, 2019.
- [19] R. J. Little and D. B. Rubin. *Statistical analysis with missing data*. John Wiley & Sons, John Wiley & Sons, 2019.
- [20] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data, 2017.
- [21] N. Patki, R. Wedge, and K. Veeramachaneni. The synthetic data vault. *2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 399-410:399--410, 2016.

- [22] D. B. Rubin. Inference and missing data. *Biometrika*, 63(3):581--592, 1976.
- [23] J. L. Schafer. *Analysis of incomplete multivariate data*. CRC press, CRC press, 1997.
- [24] R. Schnell, T. Bachteler, and J. Reiher. Privacy-preserving record linkage using bloom filters. *BMC medical informatics and decision making*, 9(1):1--11, 2009.
- [25] J. H. Seo, J. H. Cheon, and J. Katz. Constant-round multi-party private set union using reversed laurent series. In M. Fischlin, J. Buchmann, and M. Manulis, editors, *Public Key Cryptography -- PKC 2012*, pages 398--412, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [26] J. Sun, X. Yang, Y. Yao, A. Zhang, W. Gao, J. Xie, and C. Wang. Vertical federated learning without revealing intersection membership, 2021.
- [27] B. Tu, Y. Bai, C. Zhang, Y. Cao, and Y. Chen. Fast enhanced private set union in the balanced and unbalanced scenarios. *Cryptology ePrint Archive*, Paper 2025/827, 2025.
- [28] B. Tu, Y. Chen, Q. Liu, and C. Zhang. Fast unbalanced private set union from fully homomorphic encryption. *Cryptology ePrint Archive*, Paper 2022/653, 2022.
- [29] E. Uzun, S. P. Chung, V. Kolesnikov, A. Boldyreva, and W. Lee. Fuzzy labeled private set intersection with applications to private Real-Time biometric search. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 911--928, USENIX Association, Aug. 2021. USENIX Association.
- [30] F. Wang, B. Mi, and R. Zeng. Efficient private set intersection for vertical federated learning in iov. In B. Chen, X. Fu, and M. Huang, editors, *Frontiers in Cyber Security*, pages 120--130, Singapore, 2024. Springer Nature Singapore.
- [31] J. Wang, E. X. Huang, P. Duan, H. Wang, and K.-Y. Lam. Psa: Private set alignment for secure and collaborative analytics on large-scale data. *IEEE Transactions on Dependable and Secure Computing*, 23(2):2072--2084, 2026.
- [32] J. Wen, Z. Zhang, Y. Lan, Z. Cui, J. Cai, and W. Zhang. A survey on federated learning: challenges and applications. *International journal of machine learning and cybernetics*, 14(2):513--535, 2023.
- [33] Y. Xi, Y. Guo, S. Xu, C. Cai, and X. Jia. Private sample alignment for vertical federated learning: An efficient and reliable realization. *IEEE Transactions on Information Forensics and Security*, 20:3834--3848, 2025.
- [34] Q. Yang, Y. Liu, Y. Cheng, Y. Kang, T. Chen, and H. Yu. *Horizontal Federated Learning*, pages 49--67. Springer International Publishing, Cham, 2020.
- [35] C. Zhang, Y. Chen, W. Liu, L. Peng, M. Hao, A. Wang, and X. Wang. Unbalanced private set union with reduced computation and communication. *Cryptology ePrint Archive*, Paper 2024/1340, 2024.
- [36] C. Zhang, Y. Chen, W. Liu, M. Zhang, and D. Lin. Linear private set union from Multi-Query reverse private membership test. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 337--354, Anaheim, CA, Aug. 2023. USENIX Association.
- [37] Z. Zhao, X. Liang, H. Huang, and K. Wang. Deep federated learning hybrid optimization model based on encrypted aligned data. *Pattern Recognition*, 148:110193, 2024.